



Enjoy! Java Learning

ゼロから始める Javaプログラミング

シリーズ全体の目標:

新入社員がJavaプログラミングの基本的な「読み・書き」をできるようになること。単なる暗記ではなく、「なぜそうするのか」を理解し、自走できるエンジニアの第一歩を踏み出すことを目指します。



Javaで出来ること

Java 言語は、C ,Ruby, Python言語と同じような汎用言語の一つ。

さまざまなプログラムを作成することができます

- 文字情報を使用した CUI (Character-based User Interface) アプリケーション
- ウィンドウシステムを利用した GUI (Graphical User Interface) アプリケーション
- サーバ上で動作する Web アプリケーション
- Android スマフォ・Padで動作するアプリケーションやゲーム
- ミドルウェアとしての通信モジュール
- バッチ処理 (データの一括処理) モジュール などなど



この教育カリキュラム

『ゼロから始めるJavaプログラミング』は、～

以下の2つの教育プログラムからなっています。

- 「Javaプログラミング基礎編」

プログラミング初心者のため、他の言語とも共通するJavaプログラミングの構造や考え方について学ぼう！

- 「Javaオブジェクト指向プログラミング編」

Javaの最も強力な特徴であるオブジェクト指向プログラミング（OOP）と、Javaの特徴的な機能について学ぼう！

その他、実践編として

『RPGに学ぶJavaプログラミング』があります。 ※ RPG：ロールプレイングゲーム



Enjoy! Java Learning



このビデオスはJavaの基本となる文法についてまずは基礎的な概念を学ぶ構成になっています。細かな文法については、副読本には網羅的に詳しく記載があります。是非参照下さい。

また、コードサンプルについて記載がありますので、それらを参考に自分でタイピングして書いたソースコードを実際に動かしてみることをお勧めします。

第0回 Javaの世界へようこそ！

最初の一步：環境構築とプログラムの実行



この回の目標: Javaプログラミングを始めるための準備を整え、ソースコードを書いてから実行するまでの一連の流れを理解すること。

Javaプログラミング基礎編

第0回 最初の一步：環境構築とプログラムの実行 (1/5)



プログラミングの前に必要なもの：JDK

Javaプログラムを動かすには、
JDK (Java Development Kit) という開発ツールが必要です。
これは、Javaプログラムを作成する為の万能ツールボックスのようなもの

- コンパイラ

人間が書いたJavaコード (.javaファイル) を、コンピュータが理解できる言葉 (.classファイル) に翻訳してくれます。

- 実行環境

翻訳された.classファイルを実際に動かしてくれます。

実行方法① コマンドライン (基本)

プログラムが動く仕組みや手順を理解しましょう！

1. 書く  : テキストエディタでコードを書き、App.javaという名前で保存します。
2. 翻訳する  : コンパイル・コマンドラインで `javac App.java` と入力します。
App.class というファイルが作られます。
3. 実行する  : コマンドラインで `java App` と入力します。

実行方法② 統合開発環境（IDE）（推奨）

実際の開発では、IDEという専用のソフトウェアを使うのが一般的です。

- IDE:

コードを書くための高機能なエディタ、コンパイラ、デバッグツールなどが一つになったプログラミングの万能ツールボックスです。

- 主なIDE:

Eclipse、IntelliJ IDEA、Visual Studio Code(VSCode)などがあります。

- メリット

面倒なコンパイルや実行は、
再生ボタン▶を押すだけで自動でやってくれます。

まとめ

- JDKがJavaプログラミングの必須ツール。
- プログラムは 書く → 翻訳（コンパイル） → 実行 の流れで動く。
- コマンドラインはその基本手順
- IDEは開発を強力にサポートしてくれる推奨ツール。

さあ、準備は整いました！次のビデオから、いよいよJavaのコードを書いていきましょう！

第1回 Javaの世界へようこそ！ プログラムの第一歩

最初のプログラムを動かしてみよう



この回の目標:

Javaプログラムがどのような構造で動いているのか、その全体像を掴むこと。そして、お決まりの「Hello World!」を自分の手で動かし、プログラミングの楽しさを体感すること。

第1回 最初のプログラムを動かしてみよう (1/6)



プログラムの基本的な構造

Javaのプログラムは、クラス (Class) という部品が集まりでできています。

まずは、プログラムを実行するための「玄関」となる特別な場所、mainメソッド を覚えるところから始めましょう。

```
public class Main {  
    // プログラムはここから始まる！  
    public static void main(String[] args) {  
        // ここに実行したい命令を書く  
        . . . . .  
    }  
}
```

ポイント：

- class Main { ... } : Mainという名前のクラス (部品) を定義しています。
- main(...) { ... } : プログラムが起動したときに、最初
に実行される特別なメソッド (処理の塊) です。

最初のプログラム "Hello World!"

```
// Main.java
public class Main {
    public static void main(String[] args) {
        // "Hello World!" と画面に表示して改行する命令
        System.out.println("Hello World!");
    }
}
```

****実行結果：****

Hello World!

- System.out.println(...) は、括弧の中身を画面に表示するための命令です。
- 命令の最後には必ず セミコロン (;) を付けます。これを****文 (Statement) ****と呼びます。

文 (Statement) とは？

文 (Statement) とは？

- プログラムは、コンピュータに対する命令の集まりです。
- Javaでは、その一つ一つの命令を 文 (Statement) と呼び、文の終わりには必ず セミコロンの (;) を付けます。

```
// これは一つの文
System.out.println("Hello!");
// これも一つの文
int price = 100;
```

Point :

- セミコロンを忘れると、文の終わりが分からずにエラーになります。よくある間違いなので気を付けましょう！

コメントでメモを残そう

コメントとは、プログラムの動作に影響を与えない、人間のためのメモです。コードが何をしているのかを分かりやすくするために使います。

```
public class Main {  
    public static void main(String[] args) {  
        /*  
        * System.out.println は、画面に文字を表示する命令です。  
        * これから何度も使うことになるので覚えておきましょう。  
        */  
        System.out.println("Hello World!"); // "Hello World!"と表示  
    }  
}
```

コメントの書き方：

- ・ `/* ... */` : `/*` から `*/` までで囲まれた範囲をコメントにする (複数行コメント)

- ・ `//` : その行の最後までをコメントにする (一行コメント)

第1回のまとめ

- Javaプログラムは クラス (Class) の中に記述する。
- プログラムの開始点は mainメソッド。
- `System.out.println("...")` は画面に文字を表示する命令。
- 一つ一つの命令を 文 (Statement) と呼び、最後には セミコロンの (;) が必要。
- コメント を使って、コードにメモを残すことができる。

第2回 データを入れる魔法の箱！ 変数とデータ型

プログラムに情報を記憶させよう



この回の目標:

プログラムで数値や文字といった「データ」を扱うための基本である、「変数」と「データ型」の概念を理解すること。コンピュータのメモリ上にデータを保存し、それらを名前呼び出して利用する方法を学びます。

第2回プログラムに情報を記憶させよう (1/11)



『変数』ってなんだろう？

変数とは、データを入れておくための 名前付きの「箱」 のようなものです。

プログラムは、この「箱」に必要な情報を一時的に保管し、後から名前を取り出して使ったり、中身を書き換えたりします。

1. 箱を用意する (変数を宣言する)
2. 箱に名前をつける (変数名)
3. 箱にデータを入れる (値を代入する)

変数の使い方

変数を使うには、まず「こんな箱を使いますよ」と宣言する必要があります。

// 1. 変数の宣言 (箱の用意)

```
int myNum;
```

// 2. 値の代入 (箱にデータを入れる)

```
myNum = 10;
```

```
中身を表示してみる System.out.println(myNum); // 結果: 10
```

宣言と代入は、同時に行うこともできます。

// 宣言と代入を同時に行う

```
int myNum = 15;
```

```
System.out.println(myNum); // 結果: 15
```

なぜ『データ型』が必要なの？

Javaでは、変数（箱）を用意するときに、
どんな種類のデータを入れるための箱なのかをあらかじめ決めておく必要
があります。これを データ型 といいます。

- 整数を入れる箱 (int)
- 文字を入れる箱 (char)
- 文章を入れる箱 (String)

なぜ？

データの種類によって、必要なメモリのサイズや扱い方が異なるためで
す。型を決めておくことで、コンピュータは効率よく動作でき、プログラ
マの間違いも減らせます。

基本データ型① (数値)

Javaには、基本的なデータ型がいくつか用意されています。まずは数値を扱う型です。

データ型	説明	例
int	整数の最も一般的な型	10, -5, 0
double	少数の最も一般的な型	3.14, -0.01

他にもこんな仲間が…

- byte, short, long: : intよりも小さい、または大きい整数を扱う型
- float : doubleよりも精度の低い (軽い) 小数を扱う型

Point:

まずは 整数ならint、小数ならdouble と覚えておけばOK !

基本データ型② (その他)

数値以外にも、よく使う基本的なデータ型があります。

データ型 説明 例

- boolean 真偽値。「はい」か「いいえ」の2択 true or false
- char 1文字だけを扱う型 'A', 'あ', '1'
- Google スプレッドシートにエクスポート

Point:

- booleanは、条件分岐（もし〇〇だったら…）などで非常に重要になります。
- charは、シングルクォーテーション '?' で囲みます。

特別なデータ型：String

特別なデータ型：String

「Hello World!」のような**文字列（文章）を扱うには、String**型を使います。

```
String name = "John";  
String message = "This is a pen.";
```

Point:

- 厳密には基本データ型ではありませんが、非常によく使うのでセットで覚えましょう。
- 文字列は、ダブルクォーテーション "" で囲みます。

使ってみよう！

これまで学んだデータ型を使って、色々な変数を宣言してみましょう。

```
public class Variables {  
    public static void main(String[] args) {  
        // 文字列型  
        String name = "John";  
        System.out.println("My name is " + name);  
        // 整数型  
        int age = 20;  
        System.out.println("I am " + age + " years old.");  
        // 小数型  
        double height = 175.5;  
        System.out.println("My height is " + height + "cm.");  
        // 文字型  
        char bloodType = 'A';  
        System.out.println("Blood type is " + bloodType);  
        // 真偽値型  
        boolean isStudent = true;  
        System.out.println("Am I a student? " + isStudent);  
    }  
}
```

- `+` 記号を使うと、文字列と変数を連結して表示できます。

値の書き方（リテラル）

ソースコードに直接書く100や"Hello"のような値そのものをリテラルと呼びます。

Javaには、リテラルの書き方にもルールがあります。

// 123 は int型として扱われる

```
int i = 123;
```

// 3.14 は double型として扱われる

```
double d = 3.14;
```

// long型やfloat型を明示したい場合は、末尾に記号を付ける

```
long l = 123456789012L; // 'L' を付けて long型だと示す
```

```
float f = 3.14F; // 'F' を付けて float型だと示す
```

変わらない値 (定数)

プログラムの中で、
一度値を入れたら絶対に変わらないようにしたい値には、final
を付けて 定数 にします。

Java

// final を付けると、もう値を変更できなくなる

```
final double PI = 3.14;
```

PI = 3.14159; // エラーになる！ (再代入できない)

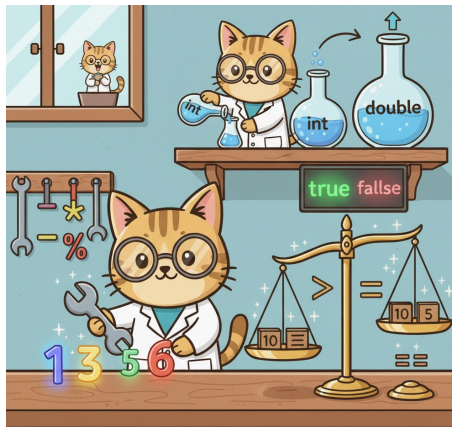
```
final int DAYS_IN_WEEK = 7;
```

第2回のまとめ

- 変数は、データを入れておくための名前付きの箱。
- データ型は、その箱にどんな種類のデータが入るかを決めるもの。
- まずは基本の5つを覚えよう！
- int (整数), double (小数), boolean (真偽), char (1文字), String (文字列)
- = は代入を意味する。
- final を付けると、値を変更できない 定数 になる。

第3回 計算と比較はお任せ！ 演算子と型キャスト

プログラムに計算をさせてみよう



この回の目標:

変数に代入したデータを、足したり引いたり、大小を比べたりする方法を学ぶこと。計算の主役である「演算子」の様々な種類と使い方をマスターし、異なるデータ型同士で計算する際の「型キャスト」という重要なルールを理解します。

第3回 プログラムに計算をさせてみよう (1/11)



違う型のデータを混ぜると？

整数(int)と小数(double)を足し算すると、結果はどうなるでしょう？

```
int a = 10;
```

```
double b = 3.5;
```

```
??? result = a + b; // resultの型は何になる？
```

Javaのルール：

異なる型同士の計算では、より表現能力の高い（大きい）型に自動的に合わせてから計算される。

$\text{int}(10) + \text{double}(3.5) \rightarrow \text{double}(10.0) + \text{double}(3.5) = \text{double}(13.5)$ そのため、resultの型はdoubleにする必要があります。

型キャスト① 自動で変換（拡大変換）

小さい型から大きい型への代入は、自動的行われます。これを**拡大変換（Widening Casting）**と呼びます。

```
int myInt = 10;  
// int から double への変換は自動で行われる  
double myDouble = myInt;  
System.out.println(myDouble); // 結果: 10.0
```

Point:

小さい箱の中身を大きい箱に入れるイメージ。データが失われる心配がないので安全です。

型キャスト② 手動で変換 (縮小変換)

大きい型から小さい型へ代入するには、手動で型変換を指示する必要があります。これを縮小変換 (Narrowing Casting) ******と呼びます。

```
double myDouble = 9.78;  
// (int)と書くことで、double型をint型へ強制的に変換  
int myInt = (int) myDouble;  
System.out.println(myInt); // 結果: 9
```

****注意！****

小数点以下のデータが失われてしまいます (**オーバーフロー**や**桁あふれ**の原因)。プログラマが「問題ない」と意志を示すために、` (int)` のように明示的なキャストが必要です。

演算子ってなんだろう？

演算子とは、計算や比較など、何らかの処理を行うための記号のことです。

Javaには様々な演算子が用意されています。

- 算術演算子: +, -, *, / ... 足し算や引き算など
- 関係（比較）演算子: ==, >, < ... 等しいか、大きいかなどを比較
- 論理演算子: &&, || ... 複数の条件を組み合わせる
- 代入演算子: =, += ... 変数に値を代入する

算術演算子

四則演算など、数学的な計算を行うための演算子です。

演算子	名称	例	説明
+	加算	$x + y$	足し算。文字列の場合は連結
-	減算	$x - y$	引き算
*	乗算	$x * y$	掛け算
/	除算	x / y	割り算
%	剰余 (じょうよ)	$x \% y$	x を y で割った余りを求める

```
System.out.println(10 / 3); // 結果: 3 (int同士の割り算は小数点以下が切り捨て)
```

```
System.out.println(10.0 / 3); // 結果: 3.333... (doubleが含まれると小数で計算)
```

```
System.out.println(10 % 3); // 結果: 1 (10を3で割った余りは1)
```

関係（比較）演算子

2つの値を比較し、その結果を boolean型 (true or false) で返します。

演算子	名称	例	説明
==	等しい	<code>x == y</code>	<code>x</code> と <code>y</code> が等しい場合にtrue
!=	等しくない	<code>x != y</code>	<code>x</code> と <code>y</code> が等しくない場合にtrue
>	より大きい	<code>x > y</code>	<code>x</code> が <code>y</code> より大きい場合にtrue
<	より小さい	<code>x < y</code>	<code>x</code> が <code>y</code> より小さい場合にtrue
>=	以上	<code>x >= y</code>	<code>x</code> が <code>y</code> 以上の場合にtrue
<=	以下	<code>x <= y</code>	<code>x</code> が <code>y</code> 以下の場合にtrue

```
int score = 85;  
boolean isPass = (score >= 80); // scoreは80以上か？  
System.out.println(isPass);    // 結果: true  
Point:
```

代入の=と、比較の==を間違えないように注意！

論理演算子

boolean型の値を組み合わせ、より複雑な条件を作るための演算子です。

演算子	名称	例	説明
-----	----	---	----

&&	AND (かつ)	cond1 && cond2	両方がtrueのときだけtrueになる
----	----------	----------------	---------------------

,		OR (または)	
---	--	----------	--

!	NOT (ではない)	!cond	trueとfalseを反転させる
---	------------	-------	------------------

```
int age = 25;
```

```
boolean hasLicense = true;
```

```
// 年齢が20歳以上 "かつ" 免許を持っているか？
```

```
boolean canDrive = (age >= 20) && (hasLicense == true);
```

```
System.out.println(canDrive); // 結果: true
```

代入演算子

右辺の値を左辺の変数に代入します。計算と代入を組み合わせた便利な短縮形もあります。

演算子	名称	例	$x = x + y$ と同じ意味
=	代入	$x = y$	-
+=	加算代入	$x += y$	$x = x + y$
-=	減算代入	$x -= y$	$x = x - y$
*=	乗算代入	$x *= y$	$x = x * y$
/=	除算代入	$x /= y$	$x = x / y$

```
int score = 100;  
score = score + 10; // score は 110 になる  
// 上の行は、複合代入演算子を使うと、こう書ける  
score += 10; // score は 120 になる
```

インクリメント・デクリメント

変数の値を1だけ増やしたり、減らしたりするための特別な演算子です。

演算子 名称 例 意味

++ インクリメント $x++$ $x = x + 1$

-- デクリメント $x--$ $x = x - 1$

```
int count = 0;
```

```
count++; // count が 1 になる
```

```
count++; // count が 2 になる
```

```
System.out.println(count); // 結果: 2
```

少しトリッキーな話（前置と後置）

++を前に置くか(++x)、後ろに置くか(x++)で、代入や比較と組み合わせた際の評価の順番が変わります。まずはx++という書き方に慣れましょう。

第3回のまとめ

- 異なる型同士の計算では、大きい型に自動で揃えられる（暗黙の型キャスト）。
- 大きい型から小さい型への変換は（型）を使って明示的に行う（明示的な型キャスト）。
- 算術演算子（+, -, /, %...）で計算ができる。
- 関係演算子（==, >, !=...）で比較ができ、結果はbooleanになる。
- 論理演算子（&&, ||）で複雑な条件を組み立てられる。

第4回 場合分けを考えよう！ 条件分岐 if文

プログラムの流れをコントロールする



この回の目標:

プログラムの実行の流れを、状況に応じて変える方法を学ぶこと。前回学んだ比較の結果 (true or false) を使い、プログラムに「もし〇〇だったら、この処理をする」という判断をさせるための「if文」をマスターします。

第4回 プログラムの流れをコントロールする (1/8)



なぜ判断が必要なの？（制御構文）

プログラムは、状況に応じて実行する処理を変える必要があります。

- テストの点数が80点以上なら「合格」、そうでなければ「不合格」と表示する。
- 年齢が20歳未満なら、お酒の購入ボタンを非表示にする。
- ログインIDとパスワードが正しければ、マイページに遷移させる。

このようなプログラムの流れをコントロールする仕組みを 制御構文（せいぎょこうぶん） と呼び、if文はその代表格です。

基本のif文

if文は、()の中の条件式がtrueの場合にのみ、{}ブロックの中の処理を実行します。

// もし (条件式) が true なら { ... } を実行する

```
if (条件式) {
```

```
    // 条件式が true の場合に実行される処理
```

```
}
```

例：

```
int score = 90;
```

```
if (score >= 80) {
```

```
    System.out.println("素晴らしい！合格です！");
```

```
}
```

`score`は`90`なので`score >= 80`は`true`。そのため、メッセージが表示されます。
もし`score`が`70`だったら、この`if`文は何も実行しません。

if-else文：「もし～でなければ」

if-else文：「もし～でなければ」

条件がfalseだった場合の処理も書きたいときは、else を使います。

```
if (条件式) {  
    // 条件式が true の場合に実行される処理  
} else {  
    // 条件式が false の場合に実行される処理  
}
```

例：

```
int score = 75;  
if (score >= 80) {  
    System.out.println("合格です！");  
} else {  
    System.out.println("残念、不合格です。");  
}
```

`score`は`75`なので`score >= 80`は`false`。そのため、**`else`ブロック**の中の処理が実行されます。

if-else if-else文：3つ以上の選択肢

選択肢が3つ以上ある場合は、else if を使って条件を追加できます。

Java

```
if (条件式1) {  
    // 条件式1が true の場合の処理  
} else if (条件式2) {  
    // 条件式1が false で、条件式2が true の場合の処理  
} else {  
    // すべての条件が false だった場合の処理  
}
```

Point: 上から順番にチェックされ、最初にtrueになったブロックだけが実行されます。残りはすべて無視されます。

使ってみよう！

if-else if-else を使って、成績を判定するプログラムを書いてみましょう。

```
int score = 85;
if (score >= 90) {
    System.out.println("素晴らしい！評価は「優」です。");
} else if (score >= 80) {
    System.out.println("良いですね！評価は「良」です。");
} else if (score >= 60) {
    System.out.println("もう少し！評価は「可」です。");
} else { System.out.println("残念…評価は「不可」です。");
}
```

****実行結果：****

良いですね！評価は「良」です。

なぜ？

1. score >= 90 (85 >= 90) → false。次へ。
2. score >= 80 (85 >= 80) → true！
3. のブロックを実行し、if文全体を終了。

中括弧 {} の役割

中括弧 {} の役割

ifやelseの後に実行する処理が1行だけの場合、中括弧 {} は省略できます。

// 1行なのでOK

```
if (score >= 80)
```

```
    System.out.println("合格です！");
```

しかし、非推奨です！

後から処理を追加した際に、{}を付け忘れてバグの原因になることが非常に多いです。

// こう書きたかった…

```
if (score >= 80) {
```

```
    System.out.println("合格です！");
```

```
    System.out.println("おめでとう！");
```

```
    // ←この行もif文の一部にしたかった
```

```
}
```

{}を付け忘れると…

```
if (score >= 80)
```

```
    System.out.println("合格です！");
```

```
System.out.println("おめでとう！");
```

```
// ←この行はif文と無関係になり、常に実行される
```

ルール：

`if`文や`else`文を使うときは、処理が1行でも必ず`{}`を付ける** 習慣にしましょう。

第4回のまとめ

プログラムの流れをコントロールする仕組みを制御構文という。

- if (条件式) は、条件式がtrueのときに処理を実行する。
- else を使うと、条件がfalseだった場合の処理も書ける。
- else if を使うと、複数の条件を繋げて、多くの選択肢を作れる。
- if-else if-elseの連鎖は、最初にtrueになったブロックだけが実行される。
- 処理が1行でも、必ず**中括弧{ }**を付ける癖をつけよう！

第5回 もう一つの選択肢！ スマートな分岐 switch文

if文よりもスッキリ書ける場面を知ろう



この回の目標:

: if-else if文とは異なるアプローチで条件分岐を行う「switch文」を学ぶこと。どのような場面でswitch文が適しており、コードをよりクリーンで読みやすくできるのかを理解します。

第5回 スマートな分岐 switch文 (1/9)



if-else ifが読みにくい時

ある一つの変数の値が「1の場合」「2の場合」「3の場合」...のように、特定の値と一致するかどうかで処理を分けたい場合、if-else ifを何度も繰り返すと、コードが縦に長くなり読みにくくなることがあります。

```
int day = 2;
if (day == 1) {
    System.out.println("Monday");
} else if (day == 2) {
    System.out.println("Tuesday");
} else if (day == 3) {
    System.out.println("Wednesday");
} else {
    System.out.println("Some other day");
}
```

`==`での比較が何度も続いていますね。
こういう時こそ`switch`文の出番です！

switch文の基本構造

switch文の基本構造

switch文は、()の中の変数の値と一致する**case**を探し、そこから処理を開始します。

switch (変数) {

case 値1:

// 変数の値が 値1 と一致した場合の処理

break;

case 値2:

// 変数の値が 値2 と一致した場合の処理

break;

default:

// どのcaseにも一致しなかった場合の処理

break;

case:

比較したい値を指定します。「もしこの値だったら」という意味です。

break:

switch文全体の処理を終了させます。非常に重要です！

default:

どのcaseにも当てはまらなかった場合に実行されます。if文の最後のelseに相当します。

使ってみよう！

先ほどの曜日判定をswitch文で書き換えてみましょう。

```
int xx = 3;
switch (xx) {
  case 1:
    System.out.println("ONE");
    break;
  case 2:
    System.out.println("TWO");
    break;
  case 3:
    System.out.println("THREE");
    break;
  default:
    System.out.println("OTHER");
    break;
}
```

****実行結果：****

THREE

if-else ifの連続よりも、何についての分岐なのか
(xxの値について) が明確で、スッキリ見えますね。

breakを忘れると…？（フォールスルー）

breakを忘れると…？（フォールスルー）

caseにbreakがない場合、処理は下のcaseへと突き抜けて実行され続けます。これをフォールスルーと呼びます。

```
int xx = 3;
switch (xx) {
  case 1:
    System.out.println("ONE");
  case 2:
    System.out.println("TWO");
  case 3:
    System.out.println("THREE");
  default:
    System.out.println("OTHER");
}
```

****意図的に****フォールスルーを使い、
「3の場合******と******4の場合に同じ処理をしたい」
という書き方もできます。

```
case 3:
case 4: // 3と4で同じ処理
  System.out.println("THREE or FOUR");
  break;
```


String型も使える

Java 7以降、switch文はintだけでなくString型でも使えるようになりました。

Java

```
String fruit = "orange";
switch (fruit) {
    case "apple":
        System.out.println("Selected fruit is an apple.");
        break;
    case "orange":
        System.out.println("Selected fruit is an orange.");
        break;
    default:
        System.out.println("Unknown fruit.");
        break;
}
```

****実行結果：****

Selected fruit is an orange.

if (fruit.equals("...")) を繰り返すよりも、はるかに直感的です。

if vs switch

どちらを使うべきか、どうやって判断する？

if文が得意なこと

- 範囲の比較 (score >= 80)
- 複雑な条件 (age >= 20 && hasLicense == true)
- boolean値そのものの分岐 (if (isLoggedIn))

switch文が得意なこと

- 一つの変数が、特定の値（定数）と一致するかどうかの比較
- 選択肢（case）が多く、分岐の条件がシンプルな場合

結論： 複雑な条件ならif文。

一つの変数に対するシンプルな多分岐ならswitch文を検討しよう！

defaultの重要性

default節は、文法的には省略可能です。しかし、常に書くことを強く推奨します。

なぜ？

1. 想定外の値への備え: caseで想定した以外の値が来たときに、プログラムが何もせずに通り返るのを防ぎます。エラーメッセージを出すなど、意図した処理を行えます。
2. 可読性の向上: これを読んだ他のプログラマが、「想定外の値のことは、ちゃんと考えてあるな」と安心できます。

default:

```
// 例えばエラーを記録するなどの処理
System.out.println("Invalid day: " + day);
break;
```

第5回のまとめ

- switch文は、一つの変数に対する特定の値での多分岐が得意。
- case で値を指定し、処理の最後には break を忘れないようにする。
- breakを意図的に省略し、フォールスルーを利用するテクニックもある。
- どのcaseにも合致しない場合の処理は default に書く。（省略せず、常に書くのが望ましい）
- 複雑な条件や範囲指定はif文、シンプルな多分岐はswitch文と使い分けよう。

第6回 面倒な作業は自動化！ ループ処理 for, while

コンピュータの真骨頂、繰り返し処理を学ぼう



この回の目標:

同じ、または類似した処理を何度も繰り返し実行する方法を学ぶこと。プログラミングを非常にパワフルにする「ループ（繰り返し）処理」の代表格であるwhile文とfor文をマスターし、面倒な反復作業をコンピュータに任せるスキルを身につけます。

第6回 繰り返し処理を学ぼう (1/9)



なぜループが必要なの？

もし、「Hello」と5回表示したい場合、どう書きますか？

// これでも動くけど…

```
System.out.println("Hello");
```

```
System.out.println("Hello");
```

```
System.out.println("Hello");
```

```
System.out.println("Hello");
```

```
System.out.println("Hello");
```

問題点：

- 面倒くさい：100回や1万回だったら？
- 修正が大変：「Hello」を「Hi」に変えるのに5箇所も直す必要がある。
- 柔軟性がない：実行する回数を変えるのが難しい。

ループを使えば、これらの問題を一挙に解決できます！

while文：条件がtrueの間ずっと

while文は、()の中の条件式がtrueである限り、{}ブロックの中の処理を繰り返し実行します。

```
while (条件式) {  
    // 条件式が true の間、繰り返される処理  
}
```

例：「Hello」と5回表示する

```
int i = 0;           // ①カウンターを初期化  
while (i < 5) {      // ②条件をチェック (trueなら中へ)  
    System.out.println("Hello");  
    i++;             // ③カウンターを更新 (忘れると無限ループ！)  
}
```

Point: ループ内で条件式のtrue/falseが変化するような処理 (i++など) を書き忘れると、ループが永遠に終わらない無限ループに陥るので注意！

for文：回数が決まっている場合に最適

for文は、特に決まった回数だけ処理を繰り返したい場合に非常に便利なループです。「カウンターの初期化、条件式、カウンターの更新」を1行で書けます。

```
// for (①初期化; ②条件式; ③更新処理)
for (int i = 0; i < 5; i++) { // 繰り返したい処理 }
```

例：「Hello」と5回表示する

Java

```
for (int i = 0; i < 5; i++) {
    System.out.println("Hello");
}
```

実行順序：

① → (② → 中身 → ③) → (② → 中身 → ③) → ... → ②がfalseになったら終了

do-while文：最低1回は実行したい

do-while文：最低1回は実行したい

do-while文は、while文と似ていますが、条件のチェックを最後に行うのが特徴です。

そのため、ブロック内の処理が最低でも1回は必ず実行されます。

```
do {  
    // 最初に必ず1回実行され、その後、条件式が true の間繰り返される処理  
} while (条件式);
```

例：

```
int i = 5;  
do {  
    System.out.println("このメッセージは1回だけ表示されます。");  
    i++;  
} while (i < 5);
```

最初に`i`が5なので、`while`の条件は`false`です。
しかし、チェックが最後なので、`do`ブロックの中身は1回実行されます。

ループの制御① break

ループの制御① break

break文は、switch文だけでなくループ処理でも使えます。ループの条件に関わらず、ループを即座に強制終了させます。

例： 0から9まで表示するが、もしiが5になったら途中でやめる

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // ループを抜ける！  
    }  
    System.out.println(i);  
}
```

実行結果：

0
1
2
3
4

ループの制御② continue

continue文は、ループを終了させるのではなく、その回の残りの処理をスキップして、次の回のループにジャンプします。

例： 0から9まで表示するが、iが5の時だけスキップする

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        continue; // 今回の処理は飛ばして、次のループへ！  
    }  
    System.out.println(i);  
}
```

実行結果：
0 1 2 3 4 6 7 8 9
(5だけが表示されていない)

拡張for文 (for-each)

配列やコレクション（複数のデータのかたまり）のすべての要素に対して、順番に処理を行いたい場合、よりシンプルに書ける拡張for文があります。

Java

```
// 配列（同じ型のデータのリスト）
char[] array = {'A', 'B', 'C'};

// 拡張for文
// arrayの中から、1つずつ要素を取り出してchに入れ、
// ブロックを実行する
for (char ch : array) {
    System.out.println(ch);
}
```

****実行結果：****

A
B
C

Point: カウンター変数(i)が不要で、シンプルに書けます。配列については、後の回で詳しく学びます！

第6回のまとめ

- 同じ処理を繰り返す仕組みをループと呼ぶ。
- while文は、条件がtrueの間、処理を繰り返す。無限ループに注意が必要。
- for文は、決まった回数の繰り返しに最適で、カウンターの管理が1行で書ける。
- breakはループを即座に中断する。
- continueは、その回の処理だけをスキップして次に進む。

第7回 コードを整理整頓！ 再利用できる部品「メソッド」

プログラムを部品化して、見通しを良くしよう



この回の目標:

処理をひとまとまりの「部品」として定義し、名前を付けて何度も呼び出せるようにする「メソッド」の概念を理解すること。メソッドを使うことで、コードの重複をなくし、プログラム全体の構造をクリーンで管理しやすくするスキルを身につけます。

第7回プログラムを部品化して、見通しを良くしよう (1/8)



同じコード、何度も書いていませんか？

同じコード、何度も書いていませんか？

例えば、プログラムの様々な場所で、自己紹介を表示する必要があります。

```
// 処理A
System.out.println("--- Profile ---");
System.out.println("Name: John");
System.out.println("Age: 25");

// ... 何か別の処理 ...
// 処理B
System.out.println("--- Profile ---");
System.out.println("Name: John");
System.out.println("Age: 25");
```

****問題点：****

- コードの重複： 同じコードが複数箇所に存在している（DRY原則違反）。
- 修正が大変： 年齢が変わったら、すべての箇所を修正する必要がある。

メソッドとは？

メソッドとは、特定の処理をひとまとめにし、名前を付けたコードのブロックです。

料理のレシピに例えるなら、一つ一つの手順ではなく、「野菜を切る」「ソースを作る」といった、まとまった作業単位に名前を付けておくようなものです。

メリット：

- 再利用性: 一度作れば、名前を呼ぶだけで何度でも同じ処理を実行できる。
- 可読性: mainメソッドがシンプルになり、プログラム全体の流れが追いやすくなる。
- 保守性: 修正が必要な場合、そのメソッドの中身を1箇所直すだけで済む。

メソッドの定義と呼び出し

まずは、引数も戻り値もない、最もシンプルなメソッドの形です。

```
public class Main {  
    // ① メソッドの定義 (部品の作成)  
    static void printProfile() {  
        System.out.println("--- Profile ---");  
        System.out.println("Name: John");  
        System.out.println("Age: 25");  
        System.out.println("-----");  
    }  
    public static void main(String[] args) {  
        System.out.println("処理を開始します。");  
        // ② メソッドの呼び出し (部品の使用)  
        printProfile();  
        System.out.println("処理を終了します。");  
    }  
}
```

‘void’: このメソッドは、何も結果を返さない（実行するだけ）という意味です。

メソッドに情報を渡す (引数)

メソッドに情報を渡す (引数)

メソッドは、呼び出し元からデータを受け取ることができます。この受け取るための変数を**引数 (ひきすう) **と呼びます。

```
// (String name, int age) の部分が引数
static void printUserProfile(String name, int age) {
    System.out.println("--- Profile ---");
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
    System.out.println("-----");
}

public static void main(String[] args) {
    // メソッドを呼び出す際に、具体的な値を渡す
    printUserProfile("Alice", 30);
    printUserProfile("Bob", 22);
}
```

これで、色々な人のプロフィールを表示できる、汎用的なメソッドになりました！

メソッドから結果を受け取る (戻り値)

メソッドは、処理の結果を呼び出し元に返すこともできます。この返される値を戻り値 (もどりち) または戻り値と呼びます。

```
// `void`の代わりに、返す値の型(int)を書く
static int add(int x, int y) {
    int result = x + y;
    // `return`で結果を呼び出し元に返す
    return result;
}

public static void main(String[] args) {
    int answer = add(5, 3); // 戻り値を変数で受け取る
    System.out.println("5 + 3 = " + answer);
    int anotherAnswer = add(100, 200);
    System.out.println("100 + 200 = " + anotherAnswer);
}
```

結果：

5 + 3 = 8

100 + 200 = 300

*** **`return`**:** メソッドの処理を終了し、指定した値を返します。

staticって何？

これまで、私たちが作るメソッドには必ずstaticというキーワードを付けてきました。

```
static void myMethod() { ... }  
static int add(int x, int y) { ... }
```

今の時点での理解：

mainメソッドはstaticなメソッドです。そして、staticなメソッドの中から呼び出せるのは、同じstaticなメソッドだけ、というルールがあります。

staticの本当の意味は？

このキーワードの真価は、Javaの最も重要な概念である**オブジェクト指向プログラミング (OOP) **を学ぶことで明らかになります。

今は「おまじないのようなもの」と考えて、先に進みましょう！

第7回のまとめ

- メソッドとは、処理をまとめた再利用可能な部品のこと。
- void は、メソッドが戻り値を返さないことを示す。
- 引数を使うと、メソッドに外部から情報を渡せる。
- return を使うと、メソッドが呼び出し元に結果（戻り値）を返せる。
- メソッド化により、コードの重複をなくし、可読性と保守性を向上させることができる。

シリーズの終わりに：次のステップへ

全7回の研修、お疲れ様でした！ 皆さんは、Javaプログラミングの「読み・書き」の基本を身につけました。

- 変数とデータ型: データを扱うための基本
- 演算子: 計算と比較
- 制御構文 (if, switch, for, while): プログラムの流れの制御
- メソッド: 処理の部品化

ここからが、本当の冒険の始まりです！

次は、Javaの最も強力な特徴である**オブジェクト指向プログラミング (OOP)** の世界が待っています。今回学んだメソッドや変数が、クラスという設計図の中でどのように活かされていくのか、さらに深い世界を探求していきましょう！

Javaを学ぶ事のモチベーション

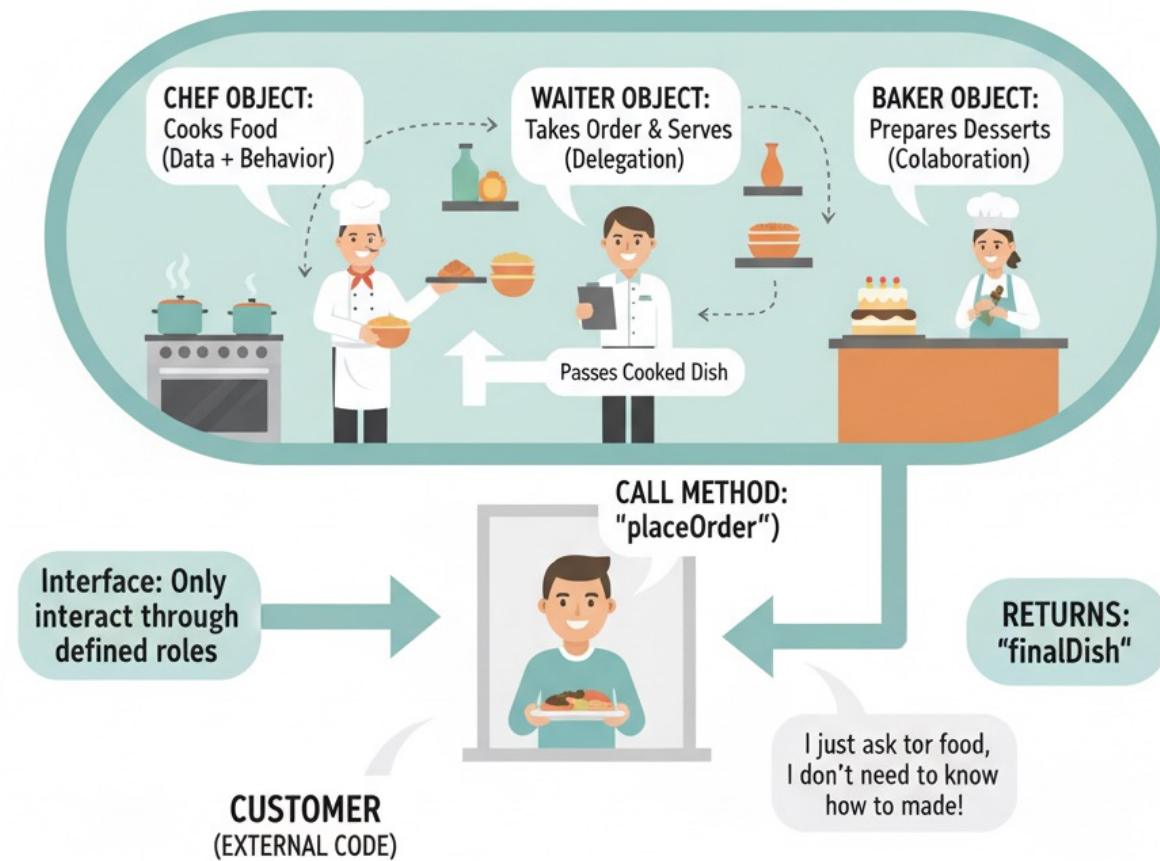
- 世の中の**重要で大規模**なシステムの多くはJavaで作られている
- Javaは**オブジェクト指向**プログラミングで、現代の多くの言語の基礎
- 歴史の中で作られてきた膨大な**エコシステム**が揃っている

エコシステム：（開発を助ける仕組み）

- Javaは**今でも進化を続けています**。ラムダ式やStream APIなど関数型プログラミングの要素も取り入れている

Javaオブジェクトシステム

RESTAURANT KITCHEN OBJECT





Enjoy! Java Learning